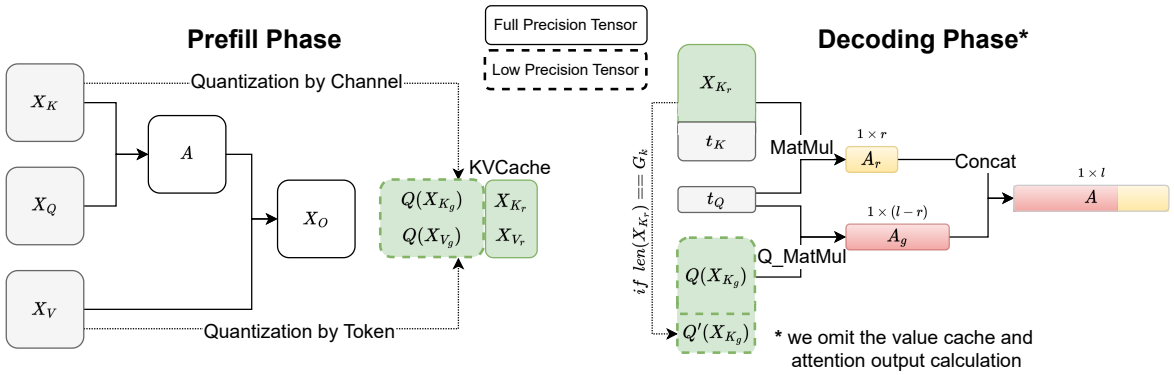


Figure 3: The overview of KIVI algorithm. For ease of illustration, we omit the value cache and attention output parts. The detailed pseudo-code is provided in Algorithm 1. Here "Q_Matmul" is the mix-precision matrix multiplication which fuses the dequantization with matrix multiplication at the tiling level.

smaller than per-channel quantization, which explains why **OB2** happens. The intuition behind this observation stems from the attention sparsity. Equation (1) can be written as:

$$[\boldsymbol{A}\boldsymbol{X}_V]_{i*} = \sum_{j=1}^{l_{\text{prompt}}} \boldsymbol{A}_{ij}[\boldsymbol{X}_V]_{j*}, \tag{2}$$

where $[\boldsymbol{X}_V]_{j*}$ is the j-th row of $\boldsymbol{X}_V$. From Equation (2), the attention output is the weighted summation of value cache across various tokens, with the weights being the attention scores. Since the attention score is highly sparse (Tian et al., 2023), the output is just the combination of value caches of a few important tokens. The per-token quantization can confine the error to each individual token. Thus, quantizing other tokens does not affect the accuracy of important tokens. Consequently, per-token quantization leads to a much smaller relative error Δ.

3.3. KIVI: Algorithm and System Support

Algorithm. As we previously analyzed, key cache should be quantized per-channel and value cache should be quantized per-token. Recall that key and value cache of newly generated tokens arrive sequentially. From the implementation perspective, per-token quantization aligns well with streaming settings, allowing newly quantized tensors to be directly appended to the existing quantized value cache by token dimension. However, for per-channel quantization, the quantization process spans across different tokens, which cannot be directly implemented in the streaming setting. As shown in Figure 3, our key idea to solve this problem is to group key cache every G tokens and quantize them separately. Because the number of tokens in $\boldsymbol{X}_K$ can be arbitrary, we split $\boldsymbol{X}_K$ into two parts, namely, the grouped part $\boldsymbol{X}_{K_g} = \boldsymbol{X}_K[:l-r]$ and residual part $\boldsymbol{X}_{K_r} = \boldsymbol{X}_K[l-r:]$, where l is the number of tokens inside the current key cache $\boldsymbol{X}_K$, r is the number of residual tokens, where $l-r$ can be divisible by G.

Since $\boldsymbol{X}_{K_g}$ can be evenly divided into $(l-r)/G$ groups, we only store $Q(\boldsymbol{X}_{K_g})$ with group-wise quantization, while $\boldsymbol{X}_{K_r}$ is kept in full precision. During the decoding process, each newly arrived key cache $\boldsymbol{t}_K$ is added to $\boldsymbol{X}_{K_r}$ and once $\boldsymbol{X}_{K_r}$ reaches R tokens, which is a hyperparameter - residual length, we quantize and concatenate it with the previously quantized $Q(\boldsymbol{X}_{K_G})$. Then we reset $\boldsymbol{X}_{K_r}$ to an empty tensor. We note that R should be divisible by G. With tiled matrix multiplication, the raw attention logits is then calculated as:

$$\begin{aligned}
\boldsymbol{A}_g &= \boldsymbol{t}_Q Q(\boldsymbol{X}_{K_g}^\top), \\
\boldsymbol{X}_{K_r} &= \text{Concat}([\boldsymbol{X}_{K_r}, \boldsymbol{t}_K]), \\
\boldsymbol{A}_r &= \boldsymbol{t}_Q \boldsymbol{X}_{K_r}^\top, \\
\boldsymbol{A} &= \text{Concat}([\boldsymbol{A}_g, \boldsymbol{A}_r]).
\end{aligned} \tag{3}$$

For value cache, similar to key cache, we also split it into two parts and keep the most recent value cache in full precision, namely, $\boldsymbol{X}_{V_g}$ and $\boldsymbol{X}_{V_r}$. Specifically, we maintain a queue and each newly arrived value cache is pushed into the queue. Once the queue reaches the predefined residual length R, the most outdated value cache is poped. Then the poped value cache is quantized per-token and concatenated with the previously quantized value cache along the token dimension.

As shown in Figure 3, we also emphasize that during the prefill phase, the exact key and value tensors are passed to the next layers, although only the quantized KV cache is retained in memory. The whole algorithm can be found in Appendix A Algorithm 1.

Analysis. In KIVI, the grouped key cache $\boldsymbol{X}_{K_g}$ and value cache $\boldsymbol{X}_{V_g}$ is quantized, while the residual key cache $\boldsymbol{X}_{K_r}$ and value cache $\boldsymbol{X}_{V_r}$ is kept in full precision. By design, there are at most R tokens inside $\boldsymbol{X}_{K_r}$ or $\boldsymbol{X}_{V_r}$. In practice, we set $R \leq 128$ and the sequence length $l_{\text{prompt}} + l_{\text{gen}}$ is often much longer than R. Thus the memory overhead from $\boldsymbol{X}_{K_r}$ and $\boldsymbol{X}_{V_r}$ is negligible when considering the benefit from extreme low-bit quantization, especially for the long context scenarios. Also, since the newly arrived key and

value tensors are added to $\boldsymbol{X}_{K_r}$ and $\boldsymbol{X}_{V_r}$ in full precision, KIVI maintains a full precision KV cache sliding window for the local relevant tokens. This window size is expected to be $\frac{R}{2}$ for key cache, and R for value cache. **Later in the experiment section, we show that this full precision sliding window is crucial for obtaining desirable performance on hard tasks, such as GSM8K.**

System Support. We provide a hardware-friendly implementation for running KIVI on GPUs. To minimize the overhead, we have fused the dequantization process with matrix multiplication, e.g., Q_MatMul in Figure 3, using CUDA. We also implement the group-wise quantization kernel in Triton. Our method is fully compatible with weight-only quantization.

4. Experiments

4.1. Settings

Models. We evaluate KIVI using three popular model families: Llama/Llama-2 (Touvron et al., 2023a;b), Falcon (Penedo et al., 2023) and Mistral (Jiang et al., 2023). Llama and Mistral model is based on multi-head attention, while Falcon is based on multi-query attention (Shazeer, 2019). We use the Hugging Face Transformers codebase and implement the KIVI algorithm upon it. Following previous work (Sheng et al., 2023), the group size G in Algorithm 1 for quantization is set as 32 across all experiments, the residual length R for key and value cache is set to 128.

Tasks. As we analyzed in Section 2, the KV cache size grows larger with a longer context. Thus, we evaluate KIVI under the normal context length and long context setting, respectively. Specifically, we adopt generation tasks from LM-Eval (Gao et al., 2021) for normal context length evaluation and LongBench (Bai et al., 2023) for long context evaluation, respectively[1]. For LM-eval, we adopt CoQA (Exact match accuracy), TruthfulQA (BLEU score), and GSM8K (Exact match accuracy). For LongBench, we chose tasks from four subgroups. Specifically, Qasper (F1 score) is a Single-Document QA task; QMSum (ROUGE score) and MultiNews (ROUGE score) are Summarization tasks; TREC (classification score), TriviaQA (F1 score), and SAMSum (ROUGE score) are Few-shot Learning tasks; and LCC (similarity score) and RepoBench-P (similarity score) is Code Completion task. The maximum sequence length in LongBench was set to 8192 for the Mistral model and 4096 for other models. We also consider the needle-in-a-haystack task (NIAH) to evaluate the model's long context retrieval ability after quantizing KV cache. Detailed NIAH setting can be found in Appendix B.

[1]The closed-end tasks such as MMLU are not ideal to evaluate KIVI since they only involve one decoding step and directly fetch the output logits, which is not suitable for studying the impact of compressed KV cache.

4.2. Accuracy and Efficiency Analysis

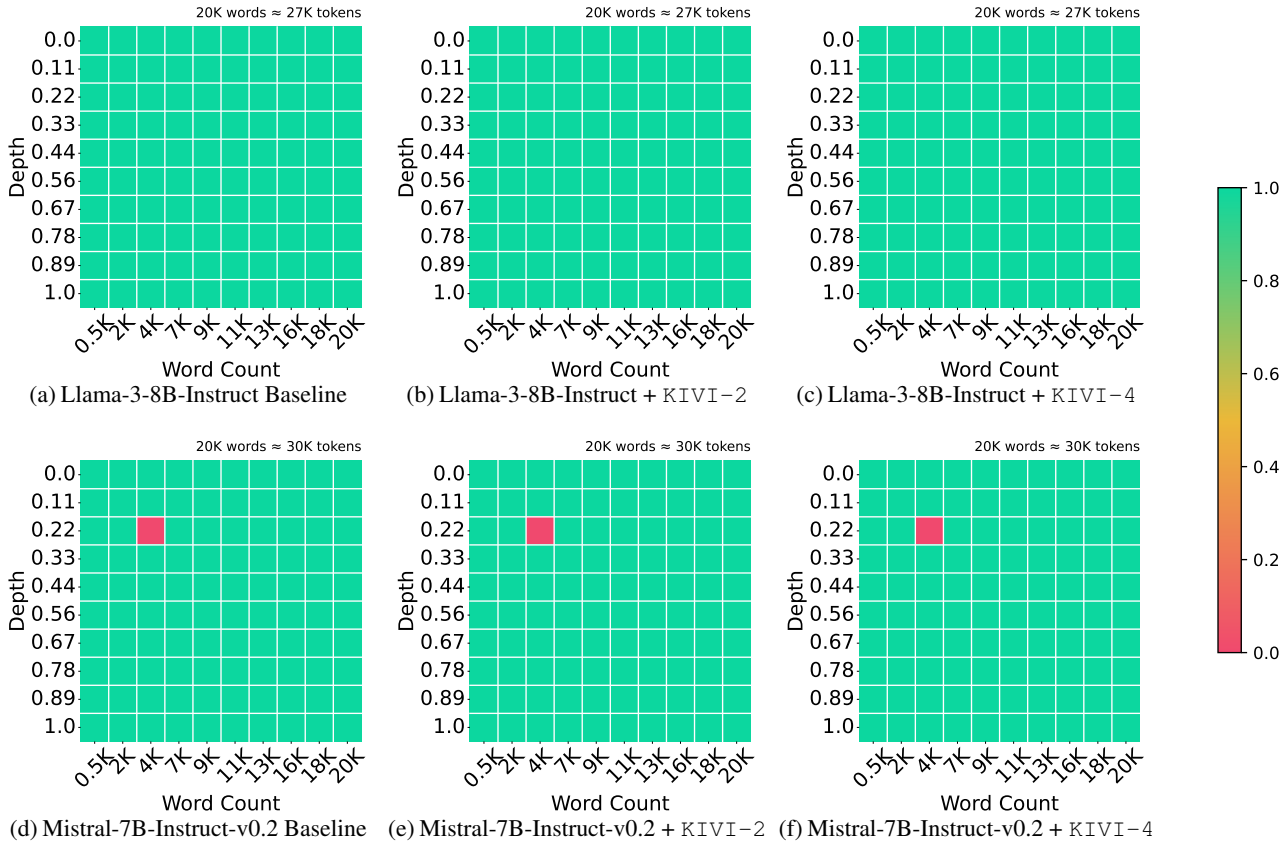
4.2.1. Comparison Between Different Quantization Configurations

We first utilize the fake quantization to demonstrate the effectiveness of our asymmetric quantization, namely, quantizing key cache per-channel and value cache per-token. Here fake quantization is exactly the same as in Table 1. The results are shown in Table 3. We observe that "2bit (K per-channel, V per-token)" consistently achieves the best results compared to all other configurations. This is consistent with our previous analysis. We also note that for hard generation tasks such as GSM8K, the fake "2bit (K per-channel, V per-token)" quantization results are significantly worse than the full precision counterparts. However, for KIVI in Table 3, we observe that the accuracy drop is only around 2% for GSM8K across different models. As we analyzed in Section 3.3, the difference between fake "2bit (K per-channel, V per-token)" quantization and KIVI is that KIVI maintains a full precision key and value cache sliding window for the local relevant tokens. This sliding window is crucial to maintaining accuracy for hard generation tasks such as mathematical reasoning.

4.2.2. Accuracy Comparison on Generation Tasks

LM-Eval Results. We benchmark KIVI in CoQA, TruthfulQA and GSM8K tasks using the LM-Eval framework. All dataset parameters were set to default. We compare the standard 16bit configuration with our KIVI compression techniques in Llama-2-7B, Llama-2-13B, Falcon-7B and Mistral-7B. As shown in Table 3, we observe that for the Llama and Mistral model, KIVI only has up to 2% accuracy drop despite the KV cache being stored in 2bit. For instance, in the Llama-2-7B model, the transition from 16bit to 2bit only slightly decreases accuracy. Similar trends are observed in other Llama-family models. Since Falcon-7B adopts multi-query attention and only has one head for KV cache, it is already highly compressed compared to Llama-based models. Thus, in Table 3, 4bit KIVI is needed to maintain the accuracy, while 2bit KIVI may have a large accuracy drop in this case.

LongBench Results. The performance of KIVI over various models in the LongBench dataset is summarised in Table 4. We apply KIVI to Llama2-7B, Llama2-13B, Llama2-7B-Chat, Llama2-13B-Chat, Falcon-7B and Mistral-7B. Table 4 suggests that KIVI is an effective method for KV cache compression with minimal impact on accuracy across var-



(a) Llama-3-8B-Instruct Baseline (b) Llama-3-8B-Instruct + KIVI-2 (c) Llama-3-8B-Instruct + KIVI-4

(d) Mistral-7B-Instruct-v0.2 Baseline (e) Mistral-7B-Instruct-v0.2 + KIVI-2 (f) Mistral-7B-Instruct-v0.2 + KIVI-4

Figure 4: Needle-in-a-Haystack results on Llama-3-8B-Instruct and Mistral-7B-Instruct-v0.2. Here we count the number of words instead of tokens to better account for the tokenizer difference. The final token length is noted in the upper right corners of each figure. Detailed setting can be found in Appendix B.

ious hard long context generation tasks. **We present additional results using Llama3-8b, Mistral-7B-v0.2, and LongChat-7B-v1.5, which can be found in Appendix D.**

NIAH Results. From Figure 4, we observe that KIVI can still maintain the retrieval ability of LLMs even with 2bit KV Cache. Detailed NIAH setting can be found in Appendix B.

4.2.3. Ablation

In this section, we benchmark KIVI on GSM8K, one of the hardest generation tasks, to show the effect of hyperparameters group size G and residual length R on the model performance. For full results of KIVI with a residual length of 32, please refer to Appendix C.

The effect of group size. We fix the residual length at 128 and vary the group sizes to 32, 64, and 128. From Table 5, we observe that group sizes 32 and 64 yield similar results, whereas the performance significantly decreases when the group size reaches 128. Note the zero-point and the scaling factor mentioned in Section 3.1 are calculated according to this group size; where the choice of group size will greatly impact the KV cache compression effect under a long input.

The effect of residual length. We fix the group size at 32 and vary the residual length across 32, 64, 96, and 128. As shown in Table 5, there is no consistent pattern between residual lengths and model accuracy. Namely, while a residual length of 128 achieves good results, 32 and 96 yield similar outcomes, but a residual length of 64 results in the worst performance. We emphasize that while we observe no significance among residual lengths of $\{32, 96, 128\}$, **having a reasonably large residual length is important; as it brings much performance boosts on hard tasks like GSM8K, again as shown in Table 5.**

4.2.4. Efficiency Comparison

To evaluate the wall-clock time efficiency of KIVI, following vLLM (Kwon et al., 2023), we synthesize workloads based on ShareGPT (sha, 2023), which contain input and output texts of real LLM services. On average, the data set has an input prompt length l_{prompt} of 161 and an output length l_{gen} of 338 (Kwon et al., 2023). We increase the batch

Table 3: Performance comparison between 16bit, 4-bit per-token quantization, four fake 2bit KV cache quantization similar to those in Table 1, KIVI-2 (2bit) / KIVI-4 (4bit) across various models. We emphasize that unlike KIVI, which preserves a small portion of full precision key cache $\boldsymbol{X}_{K_r}$ and value cache $\boldsymbol{X}_{V_r}$, all tokens in fake KV cache quantization are quantized for a fair comparison. $\mathbb{C}$ stands for per-channel quantization and $\mathbb{T}$ stands for per-token quantization.

Model		CoQA	TruthfulQA	GSM8K
Llama-2-7B	16bit	63.88	30.76	13.50
	4bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	64.82	29.85	12.28
	2bit (K - $\mathbb{C}$, V - $\mathbb{T}$)	59.08	33.10	5.76
	2bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	39.88	18.29	0.83
	2bit (K - $\mathbb{C}$, V - $\mathbb{C}$)	3.60	0.27	0.00
	2bit (K - $\mathbb{T}$, V - $\mathbb{C}$)	1.30	0.49	0.08
	KIVI-4	63.78	30.80	13.80
	KIVI-2	63.05	33.95	12.74
Llama-2-13B	16bit	66.37	29.53	22.67
	4bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	66.73	29.14	20.92
	2bit (K - $\mathbb{C}$, V - $\mathbb{T}$)	63.53	28.60	12.21
	2bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	52.93	24.98	4.55
	2bit (K - $\mathbb{C}$, V - $\mathbb{C}$)	2.88	0.74	0.00
	2bit (K - $\mathbb{T}$, V - $\mathbb{C}$)	2.80	0.26	0.08
	KIVI-4	66.38	29.49	23.65
	KIVI-2	66.23	29.84	20.77
Falcon-7B	16bit	59.83	23.20	4.55
	4bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	58.53	22.94	3.26
	2bit (K - $\mathbb{C}$, V - $\mathbb{T}$)	43.93	20.82	1.29
	2bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	25.72	0.91	0.53
	2bit (K - $\mathbb{C}$, V - $\mathbb{C}$)	41.95	17.11	1.52
	2bit (K - $\mathbb{T}$, V - $\mathbb{C}$)	19.53	0.94	0.15
	KIVI-4	59.67	22.58	4.47
	KIVI-2	57.48	24.98	3.41
Mistral-7B	16bit	67.40	30.45	38.36
	4bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	67.80	29.83	36.85
	2bit (K - $\mathbb{C}$, V - $\mathbb{T}$)	61.65	29.64	26.46
	2bit (K - $\mathbb{T}$, V - $\mathbb{T}$)	54.55	25.86	5.00
	2bit (K - $\mathbb{C}$, V - $\mathbb{C}$)	24.40	24.86	2.27
	2bit (K - $\mathbb{T}$, V - $\mathbb{C}$)	10.73	19.12	0.99
	KIVI-4	66.95	30.49	37.30
	KIVI-2	66.35	32.17	36.01

size until out of memory and report the peak memory usage and throughput between KIVI (with residual length 32 and 128) and FP16 baseline for the Llama-2-7B model. The hardware here is a single NVIDIA A100 GPU (80GB).

As shown in Figure 5, with similar maximum memory usage, KIVI enables up to $\mathbf{4}\times$ larger batch size and gives $\mathbf{2.35}\times \sim \mathbf{3.47}\times$ larger throughput. This throughput number can grow larger with longer context length and output length. We also note that **this speed-up can be greatly increased if we further fuse the KV cache quantization process with previous operations.** We leave it as one of future work.

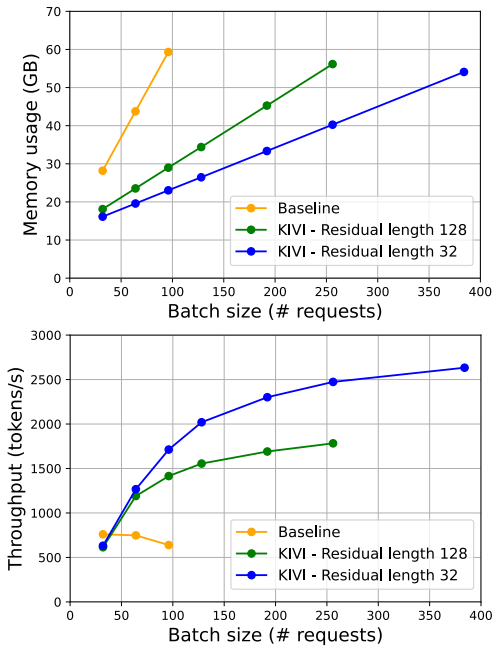


Figure 5: Memory usage and throughput comparison between 2bit KIVI and 16bit baseline. KIVI can achieve higher throughput by enabling a larger batch size.

5. Related Work

Many machine learning systems and benchmark works consider scaling up LLM inference process (Pope et al., 2023; Yuan et al., 2024). Among them, quantization techniques have been widely applied (Frantar et al., 2022; Lin et al., 2023; Kim et al., 2023; Xu et al., 2023). A main branch of LLM quantization is weight-only quantization, which involves the quantization of model weights to lower precision. For instance, AWQ (Lin et al., 2023) cleverly quantizes model weights to INT4 and INT3 using an activation-aware manner. GPTQ (Frantar et al., 2022) utilizes approximate second-order information to quantize model weights both accurately and efficiently. SqueezeLLM (Kim et al., 2023) adopts the concept of non-uniform quantization based on sensitivity along with dense-and-sparse decomposition. This line of work is orthogonal to ours, as they can be combined.

SmoothQuant (Xiao et al., 2023a) is a post-training quantization method that is more closely related to our work. This method uses equivalent transformations to balance the quantization complexity for both activation and weight, making the activation easier to quantize. SmoothQuant can compress KV cache to 8bit with minor performance loss. However, it faces a significant accuracy drop when scaled down to 4bit or less (Zhao et al., 2024). FlexGen (Sheng et al., 2023) adopts 4-bit group-wise quantization for both key and value cache.